

VarBlockSpMM: Shape-Aware CUDA Execution for Packed Variable-Block Sparse-Dense Products

Cohen Brunner Thomas
Independent Research Project

6 September 2026

Abstract

VarBlockSpMM computes FP32 sparse-dense products directly from packed variable-size blocks, avoiding scalar expansion, fixed-block retiling, output atomics, and execution workspace. On one RTX 5060 Ti, a 128-case comparison including MAGMA variable-size batched GEMM gives a geometric speedup of 1.138 over the fastest tested library, with 103 direct wins. Wide panels give the strongest advantage. A standalone variable-order discontinuous Galerkin transport program provides an application trace: at 4,096 elements and 64 simultaneous fields, 32 dependent steps run 1.438 times faster than the fastest library and 1.823 times faster than the better MAGMA composition. Smaller and narrow cases retain losses. On the separate covariance component, BSR8 wins steady state with a library-to-direct ratio of 0.390, while prepared-host startup favors direct by 3.953. Controlled experiments distinguish degree and block size effects. Device-resident inputs and value updates support reuse of the packed representation. The evidence supports choosing direct execution for the measured favorable regimes, rather than a universal replacement.

1 Introduction

Applications with dense variable blocks face a representation choice when using sparse GPU libraries. Scalar CSR expands block metadata to individual entries. Fixed-block BSR subdivides the original blocks and may require a new value layout. Grouped GEMM can retain block shapes, but overlapping contributions require separate calls or a reduction. VarBlockSpMM executes the packed product directly, so an application can retain its block layout and avoid these conversion steps.

This is useful when direct execution is faster, when scalar metadata is costly, or when a matrix is used too few times to repay library conversion and setup. We measure these benefits separately. The intended input has dense stored blocks. Artificially grouping scattered scalar nonzeros can instead waste storage and arithmetic, as the published-matrix controls show.

We study $C = AB$, where $A \in \mathbb{R}^{M \times K}$ is a packed variable-block sparse row matrix and $B \in \mathbb{R}^{K \times N}$ and $C \in \mathbb{R}^{M \times N}$ are column-major dense panels. The supported block dimensions are 8, 16, $\dots$, 64, with independently chosen row and column partitions. Dense panel widths are $N \in \{8, 16, 32, 64\}$. The implementation uses FP32 values and accumulation for non-transpose multiplication with $\alpha = 1$ and $\beta = 0$.

The implementation specializes execution by panel width while retaining runtime block dimensions. One CTA owns each block row, shares the dense input tile, and writes the completed output once. This combines a small execution plan with direct access to the application’s packed values.

The contribution is a standalone execution policy for this packed format, with an experimentally measured boundary against conversion-based library alternatives. It combines full-panel accumulation for narrow RHS widths with shape-dependent staging for wider panels, retaining one writer per output and requiring no execution workspace. The matched experiments test which choices improve this implementation. The library comparisons test when their combined benefit exceeds the benefit of padding-free BSR8 subdivision or scalar CSR expansion. These are implementation and measurement contributions. They do not establish a new sparse storage model or an end-to-end application speedup.

Startup and storage measurements separately test the cost of adopting each representation, with raw measurements tied to verified builds. Figure 1 shows where the tested widths and workloads favor direct execution.

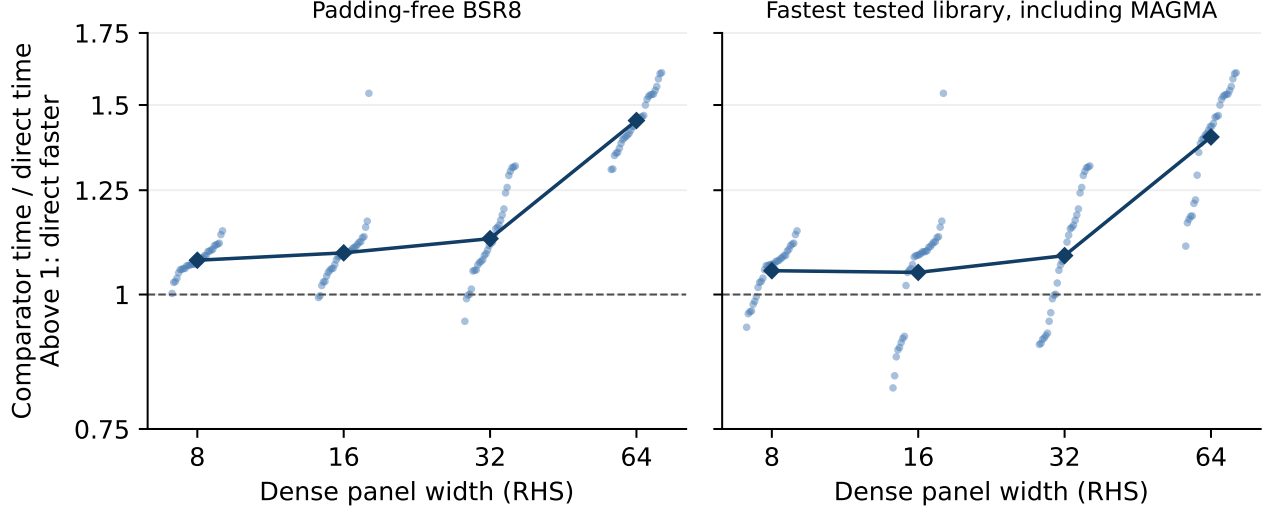


Figure 1: Direct execution by panel width on the 128-case core with MAGMA. Each point is one configuration on seed 2, measured in one process with six timing samples. Lines show geometric means. The right panel includes both MAGMA compositions and every applicable NVIDIA control. Horizontal offsets separate points. Ratios above one favor direct execution.

2 Related work

Variable blocking is established prior art. Vuduc and Moon studied variable and unaligned blocks for sparse matrix-vector multiplication [6]. OSKI defines variable block row storage through independent logical row and column partitions [7]. VarBlockSpMM uses this structural model for a dense multi-column right operand on CUDA.

GPU SpMM work has examined row assignment, load balance, instruction-level parallelism, and reuse. Yang, Buluç, and Owens describe row-splitting and merge-based approaches [1]. GE-SpMM caches sparse rows and merges warps to reduce redundant loads [8]. Tensor-core methods such as Okanovic et al. trade packing and preprocessing for dense matrix-instruction throughput [2]. These techniques motivate the present kernel. We compare the relevant MAGMA composition below, but do not compare implementations of these other research kernels.

The motivating application is adaptive construction of hierarchical H^2 matrices. Boukaram et al. describe a non-uniform batched BSR product implemented through slot-wise MAGMA non-uniform batched GEMM operations [5]. The present work addresses the underlying packed-block product. We add a generated covariance near-field component whose partitions come from a spatial cluster tree. This tests a concrete dense-block component, while excluding far-field compression and the complete construction pipeline.

NVIDIA provides scalar and fixed-block SpMM through cuSPARSE [3], and per-group GEMM dimensions through grouped cuBLAS [4]. These form the library controls. Asynchronous global-to-shared copies can overlap tile movement and arithmetic [9]. These mechanisms are established techniques. We combine and evaluate them for this particular variable-block operation.

3 Representation and kernel

Let block-row boundaries be $r_0 < \dots < r_R$, and block-column boundaries be $c_0 < \dots < c_Q$. Block (i, j) has height $h_i = r_{i+1} - r_i$ and width $w_j = c_{j+1} - c_j$. Its column-major values are packed without rounding its dimensions to a global block size, and its contribution is

$$C[r_i : r_{i+1}, :] += A_{ij} B[c_j : c_{j+1}, :].$$

Block-row pointers and column indices locate the blocks. Scalar offsets and value offsets are 64-bit. Block indices remain 32-bit. Host validation checks monotonic offsets, dimensions, index bounds, and payload sizes. A matrix owns its GPU storage and row classifications. Execution plans hold non-owning views, so the matrix must outlive its plans.

3.1 Output ownership and accumulation

One cooperative thread array, or CTA, owns one block row at every supported RHS width. Threads outside the actual row height do no output work. Each active thread accumulates all contributions to its outputs and writes them once. CTAs never update the same scalar, so execution needs neither output atomics nor partial-output reduction storage. Work imbalance between rows can still limit performance.

RHS 8 and 16 use 64-thread CTAs, with each active thread owning all RHS columns of one scalar row. RHS 32 assigns eight output columns per thread, and RHS 64 assigns sixteen. Multiple accumulators let each loaded sparse value feed independent multiply-add operations. Their register cost must be balanced against the number of resident CTAs and active threads.

Ownership and tile sequence for one block row	
Ownership	One CTA writes the entire output panel of block row i .
Initialization	Each thread sets its owned accumulators to zero.
First tile	Load the first reusable B tile into shared memory.
Pipeline step	Initiate the next tile copy into the alternate buffer, then compute the current block contribution.
Boundary	Wait for copies and synchronize the CTA before consuming the next tile or reusing a buffer.
Completion	Write each owned output once, including zero for an empty row.

Figure 2: The tall-row RHS-32 algorithm with its alternating two-buffer schedule. All supported widths retain disjoint output ownership.

3.2 Full-panel accumulation for RHS 8 and 16

The narrow-panel kernel launches one 64-thread CTA per block row. Thread $t < h_i$ owns $C[r_i + t, 0 : N]$, with N FP32 accumulators. For each stored block, all 64 threads cooperatively load its $w_j \times N$ input tile into shared memory. A barrier makes that tile visible before use. Each active thread then loads one sparse value at a time and reuses it across all N columns. A second barrier protects shared-memory reuse when another block follows.

The shared leading dimension is 65 floats. The RHS-8 and RHS-16 kernels allocate 2,080 and 4,160 shared-memory bytes per CTA, respectively. Since the largest supported row has 64 scalars, one CTA covers the entire panel without a row-size threshold or a second output tile. Empty rows write zero.

The narrow policy was selected on development seed 4 at 256, 1,024, and 4,096 block rows, degrees 2 and 16, and all four shape distributions. We compared staged kernels with different thread and accumulator counts and warp-reduction candidates, checking every output against a full CPU reference. The final policy uses full-panel accumulation and staging at both widths. Evaluation seeds are separate from this development seed. The matched campaign also measures the previous narrow policy in the same executable and process, so its improvement does not depend on comparing archived times.

3.3 Staging and dispatch

At every supported width, each local output row consumes the same dense input tile for a stored block. Cooperative staging loads this tile once per CTA and reuses it across local rows. A shared leading dimension of 65 floats reduces power-of-two bank-alignment conflicts. The synchronous RHS-32 and RHS-64 kernels use 8,320 and 16,640 shared-memory bytes per CTA, respectively. The narrow kernels use the smaller CTAs described above. RHS 32 and 64 retain their width-specific staging and buffering policies.

For RHS 32, rows of at most 16 scalars use a 128-thread single-buffer CTA. Taller rows use a 256-thread CTA with two tiles, consuming 16,640 shared-memory bytes. The next input tile is prefetched before arithmetic on the current tile. Pipeline waits and CTA barriers protect visibility and buffer reuse. A mixed-height matrix is split into two launches only when its mean block degree is at least eight. Homogeneous matrices need only the applicable launch. At lower mean degree, the mixed matrix uses the unsplit policy.

Double buffering is not enabled for RHS 64. A development experiment increased shared allocation to about 32.5 KiB per CTA and regressed a representative median from 2.891 to 4.419 ms. This observation motivated a width-specific policy rather than a universal pipeline. All dispatch thresholds were selected from development sweeps and held fixed for the main library comparison.

4 Experimental design

The main campaign measures the optimized dispatch described above against persistent library controls. All synthetic and application measurements were collected with the current implementation. The separate kernel ablations compare direct implementations under their stated protocols.

4.1 Persistent library controls

Scalar CSR. Synthetic dense blocks are expanded into scalar CSR once. Plans retain arrays, descriptors, handles, and workspace. We measure the default algorithm, explicit CSR algorithm 1 with preprocessing, algorithm 2, and algorithm 3 with preprocessing. All use FP32 compute, column-major panels, and the same non-transpose operation. Preprocessing and initial descriptor creation occur during setup or warmup, outside timing. NVIDIA documents preprocessing benefits for algorithms 1 and 3 [3].

The *CSR lower envelope* is the minimum GPU median among those four configurations within each process. Selecting on measured data favors this comparator. The envelope has no deployed tuning policy or charged tuning cost. It lets us check whether a direct-method win depends on the default library algorithm. Host ratios use a separate minimum over host medians, so they need not select the same algorithm.

Grouped cuBLAS. Blocks with equal height and width are grouped within each ordinal block slot. A slot contains at most one block from each row, so its GEMM outputs do not overlap. Slots execute sequentially. The first uses $\beta = 0$, and subsequent slots use $\beta = 1$. The handle, packed values, shape groups, and device pointer arrays persist.

The cached control refreshes input and output pointer arrays only when their base addresses change. It omits the full output clear when every row is nonempty. For matrices containing empty rows it retains the clear to initialize those rows. A separate changing-address experiment alternates between two allocated input/output pairs and includes pointer refreshes. It is paired with a direct method that alternates the same pairs. A stream-ordered CUDA kernel computes the input and output pointer arrays from persistent device offsets when needed. This avoids relying on the lifetime or staging behavior of temporary pageable host arrays. The first measured call reuses the final warmup addresses. The remaining 19 calls alternate addresses and refresh the grouped pointers. Changing-address timing includes these pointer-generation launches. The slot decomposition still incurs sequential GEMM calls and repeated output traffic, so it is a specific library composition rather than an oracle over possible grouped algorithms.

Fixed-block BSR. The uniform-32 synthetic subset is also measured with cuSPARSE BSR algorithm 1. Block values are converted to the required row-major block layout outside timing, while dense panels remain column-major. This gives a format-matched control where variable blocking is unnecessary.

Subdivided BSR8. Every supported dimension is divisible by eight. We therefore also subdivide each stored variable block into 8-by-8 blocks and measure cuSPARSE BSR algorithm 1 on every input. The scalar matrix and stored scalar count are unchanged, and no padding is introduced. A block of shape $h \times w$ becomes $(h/8)(w/8)$ fixed blocks, increasing block metadata while exposing a different scheduling and reuse tradeoff. Conversion to row-major BSR tiles occurs outside steady-state timing. The published scalar inputs retain their internal zeros in this control, whereas compact CSR removes them. BSR8 measures the effect of subdivision, while compact CSR measures the benefit of removing internal zeros.

4.2 Synthetic workloads

The core grid contains 128 configurations with 4,096 block rows and columns, degrees 2, 4, 8, 16, four panel widths, four size distributions, and two locality patterns. Uniform uses size 32. Low variance samples $\{24, 32, 40\}$, high variance samples $\{8, 16, \dots, 64\}$, and bimodal samples $\{8, 16, 48, 64\}$. Row and column dimensions are drawn independently. Every row in the core grid has exactly the requested degree. Columns are sampled without replacement and sorted. Random locality spans all block columns. Local locality samples cyclic offsets within plus or minus the requested degree. Stored values are uniform on $[-1, 1]$.

The extension adds 40 configurations. It evaluates 256, 1,024, and 4,096 block rows and columns for high-variance and bimodal sizes, random locality, all panel widths, and either constant degree 16 or the repeating degree pattern 0, 1, 16, 16. The latter has mean degree 8.25 and explicitly includes empty rows. It is constructed by retaining zero blocks, one block, or all blocks from successive rows of the degree-16 parent matrix. Eight constant-degree 4,096-row configurations already belong to the core grid. Thus the union contains 168 unique configurations.

The synthetic campaign holds matrix seed 1 fixed and repeats each configuration in three fresh processes. The dense input contains independent pseudorandom uniform values on $[-1, 1]$, generated with seed 9123. Changing-address buffers contain identical values to isolate address-dependent work.

4.3 Published matrices and partitioning

We also test BCSSTK13, BCSSTK14, and BCSSTK15 from the Harwell-Boeing collection, downloaded through SuiteSparse [10]. They originate from fluid-flow eigenvalues, the Omni Coliseum roof, and an offshore platform module, respectively [11]. Their original orders are 2,003, 1,806, and 3,948.

Symmetric Matrix Market input is expanded, duplicates are combined, and explicit zeros are removed. Values are divided by the maximum absolute entry and converted to FP32. The rows and columns are padded only at the outer boundary to the next multiple of eight. In the original scalar ordering, row partitions cycle through 8, 16, 32, 64 and column partitions through 64, 32, 16, 8. The final partition is truncated to the remaining multiple of eight. A block is stored if it contains a source nonzero. Other entries inside that block are represented as zeros.

This deterministic procedure preserves application-derived values and sparsity after normalization, but uses artificial partitions in place of native finite-element or hierarchical partitions. The measured compact-CSR controls retain only source nonzeros in the padded scalar domain, using explicit algorithms 1, 2, and 3 with preprocessing for 1 and 3. Their lower envelope is computed as above. Cached grouped cuBLAS uses the identical packed blocks as the direct method. This separates the cost of block fill from scalar sparse execution. All four panel widths and three fresh processes are retained. The partitions have 68, 61, and 132 block rows, respectively. RHS 8 and 16 use the same 64-thread staged kernel as the synthetic inputs.

4.4 Geometry-derived covariance near field

We construct the dense near-field component of a spatial covariance operator, following the bounding-box admissibility model described by Boukaram et al. [5]. Unlike grouping a pre-existing scalar sparse matrix, the spatial tree defines the dense blocks before matrix assembly.

Each input contains 2,048 or 8,192 points in three dimensions. Uniform geometry samples the unit cube. Clustered geometry places three quarters of the points in a clipped normal cloud with coordinate mean 0.3 and standard deviation 0.07, and leaves the remaining quarter uniform. Independent NumPy generator seeds 1, 2, and 3 give twelve matrices. For each matrix and panel width, three fresh processes repeat the same instance, giving 144 process observations.

A binary spatial tree splits along the longest bounding-box axis, using a stable coordinate sort and a midpoint rounded to a multiple of eight. A leaf stops at eight points, or at no more than 64 points when its bounding-box diameter is at most 0.18. Eight-point granularity is an explicit restriction of this application model. No points are padded, dropped, or grouped after scalar matrix assembly. The reordered leaf indices define both partitions. Uniform geometry at 2,048 points produces only size-eight leaves and is retained as a useful fixed-block control.

For leaf boxes s, t , let D denote Euclidean box diameter and dist the minimum Euclidean distance between boxes. We retain leaf pairs failing

$$\frac{D(s) + D(t)}{2} \leq 0.5 \text{dist}(s, t).$$

Each retained block stores the Gaussian covariance entries

$$A_{ij} = P^{-1/2} \exp(-\|x_i - x_j\|^2 / (2(0.1)^2)),$$

where P is the point count. The amplitude fixes the numerical scale across sizes. Other blocks are zero in this component. The far-field component of the full covariance operator is outside this experiment. The resulting near-field component need not itself be positive definite.

An independent scalar-row enumeration recomputes the covariance values and provides the compact-CSR CPU reference. Preparation checks every packed block against this reference exactly in FP32. Nonzero fill exceeds 99.99%, with a few FP32 underflows. This tests native spatial partitions within the specified model. It is not a production solver trace or an end-to-end hierarchical computation.

Some near-field inputs occupy a large fraction of the scalar matrix. We therefore include a persistent dense cuBLAS SGEMM control on all native and published-matrix inputs. It expands the compact scalar source to a zero-filled column-major dense matrix during startup, retains that matrix and a handle, and uses the same non-transpose operation and default cuBLAS math contract as the grouped control. Its output passes the same full-reference tolerances. Dense expansion is impractical at the main synthetic grid’s scalar dimensions and is not attempted there.

4.5 Matched mechanism experiments

The matched campaign crosses 256 and 1,024 block rows, degrees 2 and 16, high-variance and bimodal shapes, and four RHS widths with random locality. Each of the 32 configurations uses three matrix seeds and three fresh processes per seed, giving 288 processes. Both instance and process variation are therefore present, without treating timing trials as independent matrix samples.

For RHS 8, a complete $2 \times 2 \times 2$ factorial design varies linear versus eight-row output mapping, one versus two accumulators, and one versus two CTAs per block row. Every CTA has 256 threads. For wider panels, a global-load four-accumulator control and the selected wider accumulator count both use one 256-thread CTA per row. Cooperative staging is compared with direct loads using the same output mapping and CTA size. Single and double buffering use the same indirect row order, CTA size, and accumulator count. At RHS 8 and 16, an additional pair compares global and shared input loads with identical 64-thread CTAs and full-panel accumulators. The previous narrow policy and the production dispatch are measured alongside these controls. Each variant is checked against a full double-accumulating CPU reference, rather than only sampled output probes.

These comparisons isolate the named policy choices at fixed input and launch dimensions. Register allocation, instruction scheduling, and active thread count can change as consequences of a choice. They are not held artificially constant. The factorial marginal effects average across the other two RHS-8 factors, and the retained per-variant timings expose interactions rather than assuming that speedups multiply.

4.6 Timing and correctness

In the original campaigns, each method receives five warmups and 20 measured executions. Methods are randomly ordered within each process. Configuration and process-label jobs are shuffled before serial execution, with one GPU benchmark active at a time. Synthetic order uses seed 20260905, published-matrix order uses 20260906, and both supplementary campaigns use 20260907. Three process labels identify repeated observations of the same matrix, rather than three different matrix instances.

CUDA events bracket each execute call on the default stream, followed by an end-event synchronization. Synchronized host durations are also retained. The intervals measure execution calls, including any device idle time while the host submits later work. This matters particularly for library compositions with multiple calls, so the intervals do not isolate kernel-instruction time. Allocation, format conversion, initial input transfer, and correctness validation are excluded. Output buffers are poisoned before each method’s warmups, and correctness is checked both after warmup and after measurement. Every raw GPU and host duration, method position,

command, executable checksum, and process status is saved. Medians use the mean of the two middle observations for an even sample count.

The revision uses the RTX 5060 Ti with compute capability 12.0, CUDA runtime and driver API 13.4, Windows, MSVC 19.44, Release compilation, native CUDA architecture, and fast FP32 math. The Windows display driver reports version 32.0.16.1656. The GPU is not clock-locked or isolated from all operating-system activity. Per-process environment records accompany the synthetic timings.

The correctness suite covers both localities for every combination of four size distributions, four widths, and degrees 1, 4, 8, 16, giving 128 cases. Sixteen additional cases contain an empty first row. It tests explicit CSR algorithms, BSR32 and BSR8, cached pointer reuse, changed input and output addresses, and non-default streams. Deterministic tests cover all 64 block shapes at every RHS width, mixed empty rows, and completely empty matrices. NaN initialization before every comparison detects missing writes, and a deliberately empty operation must fail the comparison helper. Queued pointer changes are checked without intervening host synchronization. Every CUDA status in the test execution path is checked. Full output is compared with a CPU reference that accumulates the entire dot product in double before one final FP32 cast. Failure thresholds are maximum absolute error 5×10^{-4} or relative ℓ_2 error 5×10^{-5} , with NaN and infinity rejected.

Each timed synthetic matrix also receives 64 CPU output probes per method outside timing. Every application output is checked against a separate compact-CSR CPU reference, which also tests the packing transformation. The expanded test executable also passes Compute Sanitizer memcheck with zero reported errors. Racecheck restricted to the project’s row-owned and pointer-refresh kernels reports zero hazards. Library kernels are outside that racecheck filter. These checks do not substitute for sanitizer coverage of every timed configuration.

4.7 Startup, storage, and repeated products

On native inputs, each method records synchronized host startup time from its prepared host representation through construction, GPU transfer, library initialization or preprocessing, and one completed product. Common input and output buffers and the CUDA context already exist. Matrix storage is constructed separately for each method, and method order is randomized. This measures application-level startup within an existing CUDA process, not cold driver initialization. Correctness work is excluded.

Persistent storage counts the explicitly allocated device arrays, including matrix values and indices, row classifications, pointer and offset arrays, and the workspace requested through cuSPARSE. It excludes common dense panels, host-side metadata, allocator overhead, and opaque library-internal allocations. In particular, grouped cuBLAS storage is not presented as a measurement of its handle’s internal memory use.

For k products, we report the cost model $T_m(k) = S_m + (k - 1)H_m$, where S_m is startup including the first product and H_m is the synchronized host median for subsequent calls. A second model adds measured host geometry and representation assembly. Direct, BSR8, and grouped methods start from packed blocks, while compact CSR and the dense control use the separately timed scalar assembly. Dense expansion is included in its startup observation. The preparation script records one CPU assembly observation per matrix, not a distribution. These are amortization estimates for serial synchronized calls, not timed solver runs, CUDA-graph execution, or asynchronous throughput measurements. For each k , the comparator is the lowest total among all measured library methods. No tuning cost is charged to that lower envelope.

4.8 Aggregation and uncertainty

For configuration c , process label p , comparator b , and direct method d , let

$$y_{c,p} = \log(t_{b,c,p}/t_{d,c,p}), \quad g_c = \exp\left(\frac{1}{3} \sum_{p=1}^3 y_{c,p}\right).$$

The reported geometric mean is $G = \exp(|\mathcal{C}|^{-1} \sum_{c \in \mathcal{C}} \log g_c)$. Each configuration and each process within it has equal weight. Ratios above one favor the direct method. Minima and win counts refer to the configuration ratios g_c , not to individual timing trials.

We also report the range of three aggregate ratios obtained by holding the process label fixed. This is a descriptive repeatability check, not a confidence interval or a test of significance. Three process observations provide limited

information about tails and shared thermal drift. No population-level confidence interval is assigned to this deliberately chosen grid, and no comparison of interval widths is interpreted as a decomposition of uncertainty. Matrix-instance variation, workload selection, process noise, and hardware variation remain distinct questions.

5 Results

5.1 Choosing direct execution

Table 1 compares the complete core grid by panel width, including MAGMA. Each width contains four shape distributions, both localities, and four degrees. The MAGMA column selects the faster of its slot and reduction compositions within each process. The best-library column selects the fastest compatible method, including BSR32 on uniform blocks. These are comparator-favoring lower envelopes, not deployed tuning policies. The focused protocol and application traces are described below.

Table 1: Choosing an implementation on the 4,096-block-row core, matrix seed 2. Ratios divide comparator time by direct time. Each configuration uses one process with six samples per method.

RHS	BSR8	MAGMA best	Best library	Direct wins
8	1.076	2.357	1.052	25/32
16	1.093	2.132	1.048	24/32
32	1.127	1.598	1.087	22/32
64	1.451	2.057	1.401	32/32

Direct execution has a geometric-mean ratio of 1.138 against the fastest library and 2.016 against the faster MAGMA composition. It wins 103 of 128 best-library comparisons. All four widths have aggregate advantages, while RHS 64 wins all 32 individual comparisons. The narrow-width exceptions remain visible in the table. The original three-process NVIDIA-only comparison and its width breakdown are retained separately in the supplement.

Retaining packed blocks also avoids scalar metadata expansion and fixed-tile conversion. Section 5.6 measures the corresponding storage and startup costs. These matter when a matrix is replaced before many products can amortize a library conversion.

5.2 Original NVIDIA-library controls

The original seed-1 campaign below uses three processes per configuration and predates the MAGMA comparison. Its aggregates remain separate. Table 2 compares execution times on the core grid. The direct path has a geometric speedup of 4.262 against the CSR lower envelope and 5.757 against cached grouped cuBLAS. The respective win counts are 128 and 128 out of 128 configurations. Selecting the CSR algorithm reduces the aggregate ratio from 6.834 for default CSR to 4.262 for the lower envelope. The subdivided BSR8 ratio is 1.175, with 124 direct wins out of 128. This comparison retains block structure without padding, comparing two ways to execute the same dense blocks. The supplementary distribution figure shows the spread behind these averages. Its sorted curves expose the fastest comparators and the width of the performance range across the grid.

Table 2: Core-grid ratios over 128 configurations and three process observations per configuration. Minima and wins use GPU ratios averaged in log space across processes. Changing-address grouped execution is paired with changing-address direct execution.

Comparator	GPU ratio	Host ratio	Minimum	Wins
CSR default	6.834	6.732	4.317	128/128
CSR lower envelope	4.262	4.209	2.387	128/128
Grouped cached	5.757	5.668	2.617	128/128
Grouped changing	5.752	5.661	2.626	128/128
Subdivided BSR8	1.175	1.172	0.954	124/128

The supplementary width and degree table breaks down the results by width and degree. Each row averages the four size distributions and two localities. Direct time is the geometric mean of the corresponding process medians. Absolute times distinguish small launch-sensitive cases from larger products. The CSR column uses the lower envelope, and grouped execution uses cached pointers.

5.3 Additional seeds and queued NVIDIA comparisons

The follow-up repeats all 128 core configurations on matrix seeds 2, 3, and 5. Each seed and timing mode uses one fresh process per configuration, giving 768 processes. The two modes synchronize after either one product or eight queued products. Each process measures twenty samples, with GPU and host times divided by the number of products in the sample. The eight products use the same prepared plan and buffers. Changing-address controls alternate the two buffer pairs on each product. Validation runs outside timing, before and after the samples, and outputs are poisoned before each method. Jobs and methods are randomized, and GPU processes run serially.

Table 3 uses the same library lower envelope as the original core comparison. Each row includes 32 configurations on each of three matrix seeds. The seed range contains the smallest and largest 32-configuration geometric means. These ranges describe matrix variation combined with process noise. There is one process for each seed and mode, so they do not separate those sources or define confidence intervals. The original three-process campaign remains separate. The follow-up uses C++20 host compilation and the same library source bytes and CUDA toolkit.

Table 3: Additional matrix seeds and synchronization intervals for the NVIDIA controls. Ratios divide comparator time by direct time, so values above one favor direct execution. Batch is the number of products queued before synchronization. Wins compare direct execution with the fastest tested library for each of 96 instances.

Batch	RHS	Best library	BSR8	Wins	Seed range
1	8	1.056	1.081	75/96	1.052 to 1.060
1	16	1.046	1.091	72/96	1.045 to 1.047
1	32	1.091	1.131	65/96	1.087 to 1.094
1	64	1.395	1.443	96/96	1.393 to 1.399
8	8	1.050	1.074	73/96	1.049 to 1.052
8	16	1.034	1.079	72/96	1.032 to 1.036
8	32	1.086	1.125	64/96	1.075 to 1.104
8	64	1.392	1.452	95/96	1.384 to 1.398

Across all 384 instances per mode, the fastest-NVIDIA-library ratios are 1.139 for individual products and 1.132 for batches of eight, with 308 and 304 direct wins respectively. Every width has a geometric-mean advantage on each additional seed in both timing modes. RHS 64 gives the largest aggregate advantage, about 1.39, winning all 96 individual-product instances and 95 of 96 queued instances. The narrow-panel gains remain smaller, about three to six percent against the fastest tested library. Queueing eight products preserves this width-dependent pattern on the measured grid.

5.4 Fixed blocks, scaling, and degree imbalance

Both partitions have constant extent in uniform matrices, making fixed-block BSR a suitable control. Across the 32 uniform configurations, the BSR ratio is 0.989, with 10 direct wins. Table 4 shows the width dependence and helps distinguish variable-block reuse from the benefits already available with a uniform block format.

Table 4: Fixed-block BSR divided by direct execution on the uniform-32 subset. Each width contains eight degree and locality configurations.

RHS	BSR ratio	Minimum	Wins
8	0.982	0.951	2/8
16	0.884	0.823	0/8
32	0.925	0.903	0/8
64	1.192	1.112	8/8

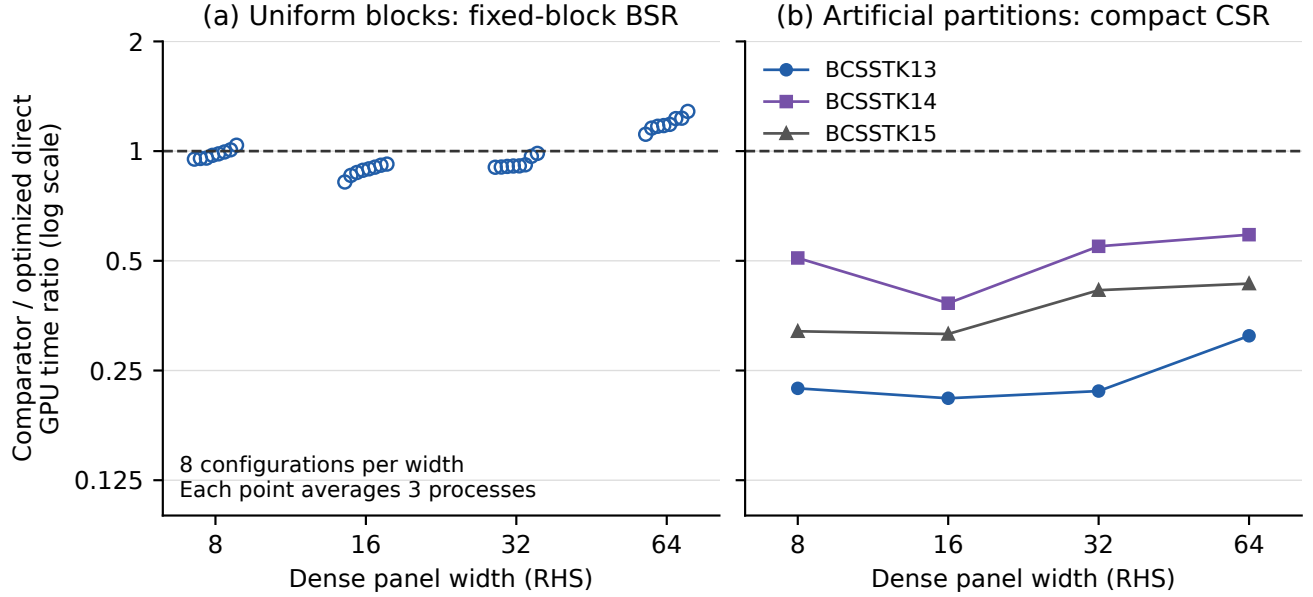


Figure 3: Format controls with a shared logarithmic ratio scale. Each point is the geometric mean of three paired process ratios. Left: all 32 uniform configurations, with eight degree and locality combinations per width. Horizontal offsets separate the points and have no numerical meaning. Right: the compact-CSR lower envelope for the twelve application configurations. Lines connect widths of the same matrix. Ratios below the dashed parity line favor the comparator. The panels represent different workloads and should not be read as a paired comparison between BSR and compact CSR.

The supplementary scaling table compares constant degree and the imbalanced pattern at three sizes. Each row includes both high-variance and bimodal dimensions and all four panel widths. The workload changes affect both total work and load balance. These results therefore test sensitivity to the combined changes and do not isolate the cost of imbalance at a fixed nonzero count. The tested maximum row degree remains 16, so heavy-tailed application degrees are not covered by this synthetic extension.

5.5 Application-derived inputs

Table 5 compares direct execution with compact CSR on the artificially partitioned published matrices. Compact CSR has an aggregate time ratio of 0.352, with 0 direct wins across twelve matrix and width configurations. Direct execution beats cached grouped cuBLAS on the same packed inputs by 5.246 in aggregate. The block-fill column shows how much of the packed storage contains source nonzeros.

Table 5: Compact-CSR lower-envelope time divided by packed direct time for published matrices under the stated artificial partitions. A ratio below one favors compact CSR. Each cell averages three process observations.

Matrix	Block fill	RHS 8	RHS 16	RHS 32	RHS 64
bcsstk13	8.9%	0.223	0.210	0.220	0.311
bcsstk14	12.2%	0.509	0.382	0.548	0.590
bcsstk15	6.9%	0.320	0.315	0.415	0.433

Low block fill makes the direct method perform arithmetic on internal zeros that compact CSR omits. These cases test the cost of choosing a packed representation for scalar sparse input. An application should retain compact CSR when its source has scattered nonzeros rather than dense blocks. Figure 3 shows these comparisons alongside uniform BSR, where the original block structure is already fixed.

5.6 Native near-field and lifecycle results

Table 6 describes the geometry-derived partitions. The clustered cases contain variable leaf sizes and much higher degrees than the synthetic grid, while the uniform cases provide small-leaf controls. The table reports overall scalar density as well as block shape. Dense SGEMM provides a useful alternative for the denser cases. All comparisons compute the identical near-field component.

Table 6: Native-model shape and degree ranges across three geometry seeds.

Points	Geometry	Leaf sizes	Block rows	Mean degree	Density %
2048	uniform	8	256 to 256	122.5 to 127.4	47.8 to 49.8
2048	clustered	8, 16, 32, 64	159 to 167	136.7 to 141.8	88.5 to 89.8
8192	uniform	8, 16	920 to 932	178.0 to 181.7	20.0 to 20.1
8192	clustered	8, 16, 32, 64	454 to 465	219.1 to 226.2	70.1 to 71.1

The aggregate BSR8 ratio on this component is 0.390, so BSR8 is faster for steady-state execution here. Direct execution offers a different benefit in startup, measured below. These matrices have many more blocks per row than the synthetic grid. Their comparison measures the cost of retaining whole block rows under that workload. Geometry, row degree, and leaf size vary together, so the two geometry families do not isolate their individual effects.

Table 7: Native-input cost models. Ratios divide the best library total by the direct total, averaging equally across input, width, and process. The assembly column adds one recorded CPU preparation observation per matrix and representation.

Products	Prepared-host ratio	Including assembly ratio
1	3.953	0.531
10	2.668	0.544
100	0.824	0.615
1000	0.412	0.491

Table 8: Explicit device-array storage ranges over native inputs and panel widths, and geometric mean synchronized startup time including one product. Storage excludes dense panels and opaque library allocations.

Method	Min MiB	Max MiB	Startup ms
direct	8.02	183.13	6.413
compact_alg1_pre	15.35	364.63	28.311
compact_alg2	15.64	371.50	40.899
compact_alg3_pre	15.33	363.99	65.078
bsr8	7.78	184.79	34.679
grouped_cached	8.85	185.84	37.324
dense	16.00	256.00	194.317

Starting from prepared host representations, direct startup has a geometric ratio of 3.953 against the fastest library alternative, including one completed product. The modeled ten-product ratio is 2.668. This measures the adoption cost when packed blocks already exist. The second column of Table 7 adds CPU assembly and therefore answers a different question about constructing the input representations.

The most compact measured scalar-CSR plan uses 1.948 times the explicit device storage of the direct representation, averaged geometrically over native inputs and widths. BSR8 and grouped GEMM retain block storage and have different metadata costs. The host assembly routines are reference implementations and have not been optimized against each other.

5.7 Matched mechanism results

The 256-thread factorial controls describe the earlier RHS-8 mapping family. The full-panel staging rows compare global and shared input loads with the same 64-thread mapping used by the final narrow kernel. These comparisons separate the contribution of staging from a change in thread count. The full factorial table is in the supplement. Per-instance and per-process method medians accompany the retained evidence.

Table 9: Previous narrow-policy time divided by final direct time, measured in the same process. Each row includes two degrees, two shape distributions, three matrix seeds, and three processes per seed. Ranges first average the three processes for each matrix instance.

RHS	Block rows	Ratio	Min	Max
8	256	1.565	1.034	1.859
8	1024	2.019	1.677	2.385
16	256	1.846	1.301	2.356
16	1024	1.699	1.392	1.999

The paired narrow-policy speedups are 1.777 at RHS 8 and 1.771 at RHS 16 across this grid. Both policies use the same input, timing protocol, and full-output reference. Development seed 4 was used to choose the replacement, while this comparison uses seeds 1, 2, and 3. Historical tuning evidence is retained in the separate supplement.

5.8 MAGMA, transport traces, and controlled factors

The focused comparison adds the FP32 variable-size batched GEMM dispatcher from MAGMA revision `c3de65f855dd` [12]. One composition follows Boukaram et al.: each batch contains at most one block per output row, and successive slots accumulate sequentially. A second composition batches all block products into temporary storage, then reduces them by output row. The latter tests whether avoiding sequential slots offsets its workspace and reduction costs. Both cache dimensions and pointers, borrow device values, and use the upstream GEMM dispatcher without kernel changes. Setup validates dimensions before calling the dispatcher used by MAGMA’s `max_nocheck` interface. Its normal launch and attribute-setting overhead remains inside timing.

The comparison repeats the 128-case core on matrix seed 2, with one fresh process and six timing samples per method. It includes the same explicit CSR algorithms, grouped cuBLAS, BSR8, and applicable BSR32 controls, plus both MAGMA compositions. This smaller follow-up is separate from the three-seed and queued campaigns. The related hierarchical study reports leaf sizes of 64 to 256, whereas this implementation supports 8 to 64 in steps of eight. The comparison therefore measures the compatible block product, not that study’s larger leaves or complete construction algorithm.

An application trace. A standalone periodic transport example solves $u_t + u_x = 0$ with an upwind discontinuous Galerkin discretization in a Legendre modal basis [13]. Prescribed element orders repeat 7, 15, 31, and 63. The resulting blocks have sizes 8, 16, 32, and 64, with dense self and upwind-neighbor couplings. Their block structure comes from the discretization, not a partition of scattered scalar entries. The program advances ensembles of sine waves through 32 forward-Euler steps using alternating device buffers. Each timing sample includes the complete dependent trace and a device copy that resets the initial state. All methods use the same FP32 operator. Every final coefficient is checked against double-precision sparse products. A separate physical L^2 check compares with the analytically translated waves. The largest relative error among the six prepared cases is below 4.2×10^{-7} .

Element counts are 256, 1,024, and 4,096, and ensemble widths are 8 and 64. Each case has three fresh processes, with six trace samples per method. The timestep is $0.05h/64^2$, where h is the element width. This tests a short transport component, not production time-to-solution or an adaptive-order solver. The full discretization and retained startup observations are described in the supplement. The comparison replaces the generic block-product backend. It does not compare DG-specific kernels that exploit the modal operator’s algebra.

On the 4,096-element, RHS-64 transport case, the fastest-library-to-direct ratio is 1.438, with 3 of three direct wins. The ratio against the faster MAGMA composition is 1.823. The table retains smaller traces and narrow ensembles, including cases that favor another implementation. This is evidence for the stated dependent transport product, not a universal application speedup.

Table 10: Complete 32-step transport traces. Ratios divide comparator time by direct time and first average paired process ratios in log space. The best-library column includes both MAGMA compositions and all tested NVIDIA controls. Times are direct GPU milliseconds per trace.

Elements	RHS	Direct ms	MAGMA best	Best library	Wins
256	8	0.372	2.560	0.828	0/3
256	64	0.822	2.071	1.052	3/3
1,024	8	0.667	4.031	1.217	3/3
1,024	64	2.893	2.175	1.209	3/3
4,096	8	3.243	2.980	1.184	3/3
4,096	64	13.357	1.823	1.438	3/3

Separating workload factors. A crossed control varies block rows over 128, 512, degree over 4, 16, 128, block size over 8, 32, and panel width over 8, 64. Each of the 24 configurations uses seed 11 and three fresh processes. Degrees select nested prefixes of the same per-row column permutation, and shape changes preserve that adjacency. The same library controls are retained, with BSR32 added where compatible. Row-count changes also change the working set, so they are not interpreted as isolated occupancy measurements. The supplementary table gives every controlled combination. These experiments distinguish the chosen factors without assigning a single cause retrospectively to the covariance result. Averaging equally over the other crossed factors, increasing degree from 4 to 128 changes the BSR8-to-direct ratio from 0.989 to 0.538 at RHS 8, and from 0.871 to 0.583 at RHS 64. Changing block size from 8 to 32 increases that ratio by factors of 1.708 and 3.206, respectively. The row-count contrast is much smaller at RHS 8 and unfavorable at RHS 64. Thus both degree and shape affect the performance boundary, and more block rows alone do not guarantee an advantage.

6 Scope and limitations

Measurements cover one GPU and one local operating environment. Three process observations describe repeatability over this campaign, while long-term drift and hardware variation require separate measurements. The original synthetic grid uses one matrix seed. The full-core follow-up adds three matrix seeds, with one process per instance and timing mode. The matched and native experiments each use three instances with three processes per instance. These chosen workloads do not define a random application population. Historical measurements use separate protocols and are excluded from the current averages.

The synthetic extension tests one degree-imbalance pattern. The published matrices have artificial partitions, and the covariance component uses eight-point partition granularity to satisfy the block-size contract. Its startup and storage results cover a generated component rather than a complete solver or hierarchical construction pipeline. Total storage and setup still include matrix values and classification metadata.

Library lower envelopes favor the comparators by selecting measured medians without charging a tuning cost. The grouped cuBLAS baseline is the stated slot composition. MAGMA is measured with both slots and a separate reduction. Other research kernels, reordering methods, other block ensembles, and additional precisions are outside this evaluation. The work contributes an implementation and measured execution policy for an established variable-block representation, with performance claims confined to the reported comparisons.

7 Reproducibility

The public API also accepts a non-owning `DeviceMatrix` when constructing a plan. Construction waits for the supplied stream and validates a host copy of metadata. Values remain on the device. Callers retain allocation ownership and keep structure fixed while the plan is in use. Device kernels may update values in place, ordered before execution. An owning matrix can instead enqueue a device-to-device value update without invalidating its plans. Structural changes require a new validated plan. The follow-up separately measures value updates and structure revalidation. It does not extend the original prepared-host startup ratios to these new input paths.

The verified Windows reference workflow uses CUDA 13.4 and MSVC 19.44. The current build uses C++20 for host and device sources. Archived receipts preserve the original compiler settings for earlier measurements. From the repository root, the retained evidence and PDFs are rebuilt with:

```
uv sync --locked
uv run --locked powershell -File scripts/build_paper.ps1
```

The build validates the archives and regenerates tables before compiling both PDFs. The README gives fresh-output commands for the five earlier campaigns. The focused follow-up has a separate builder and capped runner in `scripts/build_relevance.py` and `scripts/run_relevance.py`. Its MAGMA revision, transport input recipe, and commands are documented in `docs/relevance.md`. Python dependencies are pinned in `pyproject.toml` and `uv.lock`. Paper building also uses pdfLaTeX and pdftotext.

`scripts/build_verified.py` hashes all compiled inputs before a clean build, checks them again afterward, runs CTest, and records the resulting executable hashes and build commands. A cached receipt is reused only when every source and executable hash still matches. Each campaign snapshots the source and includes this receipt, and every process records its executable checksum. The analyzers verify the receipt against the source archive and reject missing or mismatched commands, matrix identities, design coverage, method counts, trial indices, nonfinite timings, and unsuccessful processes. The hashes verify internal consistency of the archived artifacts.

Final measurements are under `data/product-evaluation/`, in the `synthetic/`, `published/`, `native/`, and `ablation/` subdirectories, with the additional seeds and queued timings under `data/robustness/`. Generated tables and figures come from these campaigns. Native geometry, partition metadata, assembly observations, and input checksums are retained under `data/native/`. Large native binary arrays are regenerated by the preparation script and verified against their recorded checksums. Regeneration preserves the archived assembly observations. Use a fresh preparation directory to measure assembly again.

`scripts/analyze_revision.py` validates the main library campaigns. `scripts/analyze_supplement.py` validates native and matched controls. Both emit full-precision summaries beside the generated tables. `scripts/build_paper.ps1` requires both analyses to pass before compiling the manuscript. Regression tests in `tests/test_evidence.py` exercise corrupt timing records and stale source or binary receipts. The historical RHS-8 archive is validated separately for the supplement. The two seed-4 development archives are under `data/narrow-tuning/` and `data/narrow-tuning-2/`. Their 96 processes are validated by `scripts/analyze_tuning.py` and are excluded from evaluation averages. It predates build-linked receipts and is not used to generate current library ratios or matched mechanism effects.

8 Conclusion

VarBlockSpMM is a useful alternative when an application has dense variable blocks and sufficiently favorable panel widths and shapes. The core comparison including MAGMA gives a fastest-library ratio of 1.138. The large RHS-64 transport trace gives 1.438 over the fastest library, establishing a benefit for dependent products in a standalone application component.

The implementation shares input tiles, reuses sparse values across accumulators, and retains one writer per output without execution workspace. Its device-input and value-update paths let callers retain their packed allocations. Controlled results show why degree and shape belong in the choice of implementation. BSR8 remains preferable on the measured covariance component, and scalar CSR remains preferable for the published sparse inputs. The reported regimes guide the choice without claiming universal superiority.

References

- [1] C. Yang, A. Buluç, and J. D. Owens. Design Principles for Sparse Matrix Multiplication on the GPU. *Euro-Par*, 2018. <https://arxiv.org/abs/1803.08601>
- [2] P. Okanovic et al. High Performance Unstructured SpMM Computation Using Tensor Cores. 2024. <https://arxiv.org/abs/2408.11551>
- [3] NVIDIA. cuSPARSE API reference, SpMM algorithms and preprocessing. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cusparse/index.html#cusparsespmm>

- [4] NVIDIA. cuBLAS grouped batched GEMM. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cublas/#cublas-t-gemmgroupedbatched>
- [5] W. Boukaram, Y. Liu, P. Ghysels, and X. S. Li. Adaptive Sketching Based Construction of H^2 Matrices on GPUs. *IPDPSW*, 2025. <https://arxiv.org/abs/2506.16759>
- [6] R. Vuduc and H.-J. Moon. Fast Sparse Matrix-Vector Multiplication by Exploiting Variable Block Structure. *HPCC*, 2005. https://doi.org/10.1007/11557654_91
- [7] Berkeley Benchmarking and Optimization Group. OSKI Variable Block Row Format. 2007. https://bebop.cs.berkeley.edu/oski/html/group__MATTYPE__VBR.html
- [8] G. Huang, G. Dai, Y. Wang, and H. Yang. GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. *SC20*, 2020. <https://doi.org/10.1109/SC41405.2020.00076>
- [9] NVIDIA. Asynchronous Data Copies. CUDA Programming Guide. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html>
- [10] SuiteSparse Matrix Collection. HB/BCSSTK13, HB/BCSSTK14, and HB/BCSSTK15. Accessed 5 September 2026. <https://sparse.tamu.edu/HB/bcsstk13>
- [11] NIST Matrix Market. BCS Structural Engineering Matrices. BCSSTK13, BCSSTK14, and BCSSTK15 problem descriptions. <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc1/bcsstk13.html> <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc2/bcsstk14.html> <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc2/bcsstk15.html>
- [12] MAGMA developers. Variable-size batched FP32 GEMM. Revision c3de65f855dd, accessed 6 September 2026. <https://github.com/icl-utk-edu/magma>
- [13] B. Cockburn and C.-W. Shu. Runge-Kutta Discontinuous Galerkin Methods for Convection-Dominated Problems. *Journal of Scientific Computing*, 16, 2001, pages 173 to 261. <https://www.dam.brown.edu/people/shu/pub.html>